

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

Facultad de Ciencias de la Computación · Ingeniería de Software

Desarrollo de Aplicaciones Web con PHP 8

Guía completa del estudiante — con ejercicios de todos los niveles

Del primer echo a una aplicación CRUD segura con PDO y PostgreSQL

Asignatura: Aplicaciones Web (5.º semestre) — Unidad II

Docente: Dr. Gleiston Cicerón Guerrero Ulloa, Ph.D.

Período: PPA 2026–2027

Las versiones de referencia (PHP 8.4, PostgreSQL 18, Composer 2) se detallan en la sección 2.

Contenido

1	Introducción	4
1.1	¿Qué es PHP y por qué dominarlo?	4
1.2	Cómo funciona PHP en la web	4
1.3	Versiones de PHP y estado actual	4
1.4	Cómo usar esta guía y los niveles de los ejercicios	5
2	Preparación del entorno de desarrollo	5
2.1	Opciones para tener PHP funcionando	5
2.2	Instalación recomendada para empezar (XAMPP)	5
2.3	El servidor embebido de PHP para practicar rápido	6
2.4	Configurar PHP en el editor	6
2.5	Verificar la instalación	6
2.6	Estructura de un proyecto PHP	7
2.7	Versiones de referencia (verificadas a junio de 2026)	7
3	Fundamentos del lenguaje	8
3.1	Estructura de un script y las etiquetas <code><?php ?></code>	8
3.2	Variables, tipos y constantes	8
3.3	Cadenas: comillas simples vs dobles e interpolación	9
3.4	Operadores	9
3.5	Estructuras de control	10
3.6	Arrays	11
3.7	Funciones	11
3.8	Tipado estricto	12
3.9	Ejercicios — nivel básico	12
4	PHP del lado del servidor: la web dinámica	13
4.1	El ciclo de una petición	13
4.2	Las superglobales	13
4.3	Procesamiento de formularios (GET vs POST)	14
4.4	Validación y saneamiento de la entrada	14
4.5	Escapado de la salida (anti-XSS)	15

4.6	Generar HTML5 semántico desde PHP	16
4.7	Ejercicios — nivel intermedio	16
5	Organización del código	17
5.1	Incluir archivos: include y require	17
5.2	Espacios de nombres (namespaces)	17
5.3	Composer y el autoloader PSR-4	18
5.4	Manejo de errores y excepciones	18
5.5	Ejercicios — organización	19
6	Programación orientada a objetos	19
6.1	Clases, propiedades y métodos	20
6.2	Constructores y promoción de propiedades	20
6.3	Herencia, interfaces, clases abstractas y traits	21
6.4	Enumeraciones (enums)	21
6.5	Inyección de dependencias (concepto)	22
6.6	Ejercicios — OOP	22
7	Sesiones, cookies y estado	23
7.1	Cookies	23
7.2	Sesiones	23
7.3	Autenticación básica: un inicio de sesión seguro	24
7.4	El patrón Post-Redirect-Get	24
7.5	Ejercicios — sesiones y estado	25
8	Acceso a base de datos con PDO	25
8.1	¿Por qué PDO?	25
8.2	La conexión	25
8.3	Sentencias preparadas (anti-SQL injection)	26
8.4	CRUD completo	27
8.5	Transacciones	27
8.6	Ejercicios — base de datos	28
9	Proyecto integrador: CRUD de estudiantes	28
9.1	Estructura del proyecto	28

9.2	El esquema de la base	28
9.3	La conexión	29
9.4	El punto de entrada: lógica	29
9.5	El punto de entrada: presentación	30
9.6	Ejercicio integrador	31
10	Seguridad en aplicaciones PHP	32
10.1	Validar la entrada y escapar la salida	32
10.2	Inyección SQL	32
10.3	Cross-Site Scripting (XSS)	32
10.4	Falsificación de petición entre sitios (CSRF)	32
10.5	Gestión de contraseñas	33
10.6	Configuración y cabeceras seguras	33
10.7	Ejercicios — seguridad	33
11	Buenas prácticas y ecosistema moderno	34
11.1	Estándares PSR	34
11.2	Composer y Packagist	34
11.3	Frameworks: Laravel y Symfony	34
11.4	Pruebas con PHPUnit	34
11.5	Calidad de código	35
12	Frameworks de PHP: Laravel y Symfony comparados	35
12.1	¿Qué resuelve un framework y por qué comparar dos?	35
12.2	Los dos protagonistas: perfiles	36
12.3	Comparativa 1: crear el proyecto	36
12.4	Comparativa 2: una ruta y un controlador	37
12.5	Comparativa 3: el modelo y el acceso a datos (ORM)	37
12.6	Comparativa 4: las plantillas (vistas)	38
12.7	Comparativa 5: la validación	39
12.8	El CRUD de estudiantes, ahora con framework	40
12.9	Tabla comparativa y ¿cuál elegir?	41
12.10	Ejercicios — frameworks	41
13	Banco de ejercicios por nivel	42

13.1 Nivel básico	42
13.2 Nivel intermedio	42
13.3 Nivel avanzado	43
13.4 Proyectos propuestos	43
14 Soluciones seleccionadas	43
14.1 Vocales en una cadena (básico 12.1.2)	43
14.2 Pares con array_filter (básico 12.1.4)	44
14.3 API JSON de estudiantes (avanzado 12.3.1)	44
15 Glosario de siglas y términos	45
16 Recursos y referencias	46

1 Introducción

PHP es, posiblemente, el lenguaje con el que más rápido verás resultados en la web del lado del servidor, y uno de los que más encontrarás en el mundo profesional: mueve desde pequeños sitios hasta plataformas enormes. Esta guía te lleva de cero a una aplicación completa y segura, explicando cada concepto con detalle y proponiéndote ejercicios de todos los niveles para que practiques de verdad.

La estructura es progresiva: primero el lenguaje, luego su uso en la web, después la organización del código, la programación orientada a objetos, el manejo de estado, el acceso a datos y, finalmente, un proyecto integrador y la seguridad. Cada sección termina con ejercicios, y al final hay un banco de problemas por nivel con soluciones seleccionadas.

1.1 ¿Qué es PHP y por qué dominarlo?

PHP (acrónimo recursivo de «PHP: Hypertext Preprocessor») es un lenguaje interpretado, de propósito general, especialmente diseñado para el desarrollo web. Nació en 1995 de la mano de Rasmus Lerdorf y evolucionó hasta convertirse, con la serie 8, en un lenguaje moderno, rápido y con un sistema de tipos robusto. Su curva de entrada es suave, pero permite construir sistemas grandes y bien estructurados.

Dominarlo te aporta tres ventajas concretas. Primera, empleabilidad: una parte enorme de la web (gestores de contenido, tiendas, APIs) corre sobre PHP. Segunda, productividad: con muy poco código se obtiene una página dinámica. Tercera, transferencia de conceptos: los patrones que aprenderás aquí (peticiones, formularios, sesiones, acceso a datos, seguridad) son los mismos que reaparecen en Java o .NET.

1.2 Cómo funciona PHP en la web

Cuando un navegador solicita una página `.php`, el servidor web (Apache o Nginx) no la envía tal cual: la entrega al intérprete de PHP, que ejecuta el código, normalmente consulta una base de datos y produce HTML. Ese HTML es lo único que viaja de vuelta al navegador. Por eso el visitante nunca ve tu código PHP: solo ve su resultado.

Este modelo se llama petición–respuesta (request–response) y se apoya en HTTP, un protocolo sin estado: el servidor no recuerda al usuario entre una petición y la siguiente. Más adelante (sección 7) verás cómo las sesiones y las cookies resuelven esa «amnesia» para implementar, por ejemplo, un inicio de sesión.

1.3 Versiones de PHP y estado actual

PHP avanza con una versión mayor cada año aproximadamente. La serie 8 supuso un salto enorme en rendimiento y en tipado. A día de hoy (mediados de 2026) la rama estable más reciente es PHP 8.4, y 8.3 sigue siendo plenamente soportada; ambas son excelentes para aprender y producir. Conviene usar siempre una versión con soporte activo por motivos de seguridad.

En esta guía el código está pensado para PHP 8.3/8.4 y usa características modernas como el tipado estricto, la promoción de propiedades en el constructor y la expresión `match`. Cuando una

característica requiera una versión concreta, se indicará.

1.4 Cómo usar esta guía y los niveles de los ejercicios

Lee cada sección, reproduce los ejemplos en tu propio entorno y, sobre todo, resuelve los ejercicios: programar se aprende programando. Te recomendamos no copiar y pegar, sino teclear el código, porque así detectas detalles que de otro modo pasan inadvertidos.

Los ejercicios están etiquetados por nivel para que puedas dosificarlos:

- **[Básico]** afianzan la sintaxis y los conceptos esenciales; deberías poder resolverlos justo después de leer la sección.
- **[Intermedio]** combinan varios conceptos y se acercan a problemas reales (formularios, validación, objetos).
- **[Avanzado]** exigen diseño, seguridad y acceso a datos; son los más parecidos a lo que harás en un proyecto.

Al final encontrarás un banco de ejercicios por nivel y un conjunto de soluciones comentadas para que compares tu enfoque con uno posible.

2 Preparación del entorno de desarrollo

Antes de escribir código necesitas un entorno donde ejecutarlo. PHP requiere un intérprete y, para la web, un servidor que lo invoque; para las prácticas con datos, además, un servidor de base de datos. A continuación se describen las opciones y la instalación recomendada.

2.1 Opciones para tener PHP funcionando

Hay varias formas de preparar PHP, según cuánto quieras instalar y qué tan parecido a producción lo necesites. No hay una única correcta; elige según tu caso.

- **XAMPP:** un paquete todo-en-uno (Apache + PHP + MariaDB/MySQL + phpMyAdmin). Es el más sencillo para empezar en Windows.
- **Laragon:** alternativa muy cómoda en Windows, ligera y con cambio fácil de versiones de PHP.
- **Servidor embebido de PHP:** el propio PHP trae un servidor de desarrollo (`php -S`) que basta para practicar sin instalar Apache.
- **Docker:** levanta PHP, el servidor web y la base de datos en contenedores; es lo más cercano a producción, pero añade complejidad.

2.2 Instalación recomendada para empezar (XAMPP)

Para esta guía, la vía más simple en Windows es XAMPP: instala Apache y PHP juntos y te deja una carpeta `htdocs` donde colocar tus proyectos. Descárgalo del sitio de Apache Friends, instálalo y arranca Apache desde su panel de control.

Una vez instalado, tus páginas van dentro de `C:\xampp\htdocs`. Si creas `htdocs/practica/index.php`, podrás verla en el navegador en `http://localhost/practica/`. Es el modelo clásico de despliegue de PHP.

2.3 El servidor embebido de PHP para practicar rápido

Para ejercicios pequeños no necesitas Apache. Desde la carpeta de tu proyecto, en una terminal, puedes arrancar el servidor de desarrollo que trae PHP. Es ideal para iterar deprisa mientras aprendes.

Servidor de desarrollo

```
1 C:\proyecto> php -S localhost:8000
2 [PHP Development Server] started on http://localhost:8000
```

Con eso, el archivo `index.php` de esa carpeta queda disponible en `http://localhost:8000`. Para detenerlo, pulsa `Ctrl+C` en la terminal.

No es para producción

El servidor embebido (`php -S`) es solo para desarrollo: atiende una petición a la vez y carece de las protecciones de un servidor real. En producción se usa Apache o Nginx con PHP-FPM.

2.4 Configurar PHP en el editor

Puedes usar PhpStorm (el IDE de JetBrains especializado en PHP), IntelliJ IDEA con el plugin de PHP, o Visual Studio Code con extensiones de PHP. En todos los casos, el paso clave es indicarle al editor dónde está el intérprete `php.exe` para que pueda ejecutar y analizar tu código.

En PhpStorm o IntelliJ, ve a la configuración de lenguajes, sección PHP, y selecciona el intérprete (por ejemplo, `C:\xampp\php\php.exe`). A partir de ahí tendrás resaltado, autocompletado y la posibilidad de ejecutar y depurar.

2.5 Verificar la instalación

Conviene comprobar que PHP responde antes de avanzar. Hazlo de dos formas: por terminal y por navegador. Por terminal, consulta la versión instalada.

Comprobar la versión

```
1 C:\> php -v
2 PHP 8.4.x (cli) ...
3 Copyright (c) The PHP Group
```

Por navegador, crea un archivo con una sola instrucción que muestra toda la configuración de PHP; es útil para confirmar extensiones (como `pdo_pgsql`).

info.php

```
1 <?php
2 phpinfo();
```

Recuerda activar la extensión de PostgreSQL

Para los ejemplos de base de datos necesitarás la extensión `pdo_pgsql`. Verifica en `phpinfo()` que aparece; si no, actívala en `php.ini` quitando el `;` de la línea `extension=pdo_pgsql` y reinicia el servidor.

2.6 Estructura de un proyecto PHP

Un proyecto ordenado facilita el mantenimiento. Aunque al principio trabajarás con un solo archivo, conviene adoptar pronto una organización por carpetas que separe la configuración, la lógica y las vistas.

Estructura sugerida

```
1 mi_proyecto/
2 |- index.php           # punto de entrada
3 |- config/             # conexion y configuracion
4 |- src/                # clases (PSR-4, con namespaces)
5 |- vistas/             # plantillas HTML/PHP
6 |- public/             # recursos estaticos (css, js, img)
7 |- composer.json       # dependencias
```

2.7 Versiones de referencia (verificadas a junio de 2026)

Estas son las versiones vigentes de las herramientas usadas en la guía. Los paquetes de PHP se gestionan con Composer, que instala la última versión compatible; no es necesario fijarlas a mano.

Componente	Versión vigente (jun. 2026)
PHP	8.4 (estable); 8.3 con soporte activo.
PostgreSQL	18.4 (serie 18; la 19 en beta).
Composer	2.x (gestor de dependencias de PHP).
XAMPP	Edición con PHP 8.4 (Apache + MariaDB).
PhpStorm / IntelliJ	2026.1.x con plugin de PHP.
PDO / pdo_pgsql	Incluidos con PHP; activar la extensión.

3 Fundamentos del lenguaje

En esta sección recorrerás la base de PHP: cómo se escribe un script, cómo se declaran datos, cómo se controla el flujo y cómo se organizan los valores en arrays y funciones. Son los cimientos sobre los que se apoya todo lo demás, así que dedícales tiempo y resuelve los ejercicios del final.

3.1 Estructura de un script y las etiquetas `<?php ?>`

El código PHP se escribe entre las etiquetas de apertura `<?php` y de cierre `?>`. Todo lo que esté fuera de ellas se envía al navegador tal cual (texto o HTML); todo lo que esté dentro se ejecuta. En archivos que son solo PHP, se recomienda omitir la etiqueta de cierre final para evitar espacios accidentales en la salida.

Primer script

```
1 <?php
2 // Esto es un comentario de una línea
3 # También es un comentario
4 /* Y este es un comentario de varias líneas */
5 echo 'Hola, mundo'; // imprime texto
6 ?>
7 <p>Esto es HTML normal, fuera de PHP.</p>
```

La instrucción `echo` muestra texto. Cada sentencia termina en punto y coma (;). Mezclar HTML y PHP es habitual, pero más adelante verás por qué conviene separarlos en proyectos serios.

3.2 Variables, tipos y constantes

Una variable empieza por `$` y no se declara con un tipo: PHP infiere el tipo del valor que le asignas (tipado dinámico). Aun así, PHP 8 permite y recomienda declarar tipos en funciones y propiedades para evitar errores. Los tipos básicos son enteros, flotantes, cadenas, booleanos, arrays, objetos y `null`.

Variables y tipos

```
1 <?php
2 $nombre = 'Ana'; // string
3 $edad = 21; // int
4 $nota = 8.5; // float
5 $activo = true; // bool
6 $nada = null; // null
7
8 var_dump($edad); // muestra tipo y valor: int(21)
9 echo gettype($nota); // 'double'
10
11 const IVA = 0.15; // constante (no cambia)
12 define('SITIO', 'UTEQ'); // otra forma de constante
```

Usa `var_dump()` para inspeccionar una variable cuando algo no funcione: te dice su tipo y su contenido, lo que ahorra muchísimo tiempo de depuración. Las constantes, por convención, se

escriben en mayúsculas.

3.3 Cadenas: comillas simples vs dobles e interpolación

La diferencia entre comillas en PHP es idéntica en espíritu a la de Perl y es fuente de errores: las comillas dobles interpolan variables y secuencias de escape; las simples no. Esto importa especialmente con datos que contienen el símbolo \$, como ciertas contraseñas.

Comillas e interpolación

```

1 <?php
2 $nombre = 'Ana';
3 echo "Hola, $nombre";    // Hola, Ana    (interpolación)
4 echo 'Hola, $nombre';    // literal: muestra el símbolo y el nombre tal cual
5 echo "Suma: " . (2 + 3); // concatenación con el punto .
6
7 // Heredoc (interpolación, como las comillas dobles):
8 $html = <<<HTML
9 <h1>Hola, $nombre</h1>
10 HTML;
11
12 // Nowdoc (literal, como las comillas simples):
13 $texto = <<<'TXT'
14 Aquí $nombre NO se interpola.
15 TXT;
```

La contraseña con \$ y @

Si guardas una clave como `P@$7Gr3$QL` en una cadena con comillas dobles, PHP intentará interpolar `$7` y `$QL`. Escríbela siempre en comillas SIMPLES: `$clave = 'P@$7Gr3$QL';`

3.4 Operadores

Los operadores combinan valores. Además de los aritméticos y de comparación habituales, PHP tiene operadores muy útiles para el día a día, como el de concatenación de cadenas y los de fusión de nulos. Conviene conocer la diferencia entre comparación flexible y estricta.

Operador	Significado y ejemplo
.	Concatena cadenas: 'a' . 'b' da 'ab'.
== / ===	Igualdad flexible (convierte tipos) / estricta (mismo tipo y valor).
!= / !==	Distinto flexible / estricto.
&& !	Y, O, NO lógicos.
?? / ??=	Fusión de nulos: <code>\$x ?? 'def'</code> usa 'def' si <code>\$x</code> es null.

Operador	Significado y ejemplo
<code>?:</code>	Operador ternario corto (Elvis): <code>\$a ? : \$b</code> .
<code><=></code>	Nave espacial: devuelve -1, 0 o 1 (util al ordenar).

La regla de oro: usa `===` y `!==` (comparación estricta) por defecto. La comparación flexible (`==`) realiza conversiones de tipo que provocan resultados sorprendentes y errores difíciles de encontrar.

3.5 Estructuras de control

Las estructuras de control deciden qué se ejecuta y cuántas veces. PHP ofrece las clásicas (`if`, `switch`, `for`, `while`, `foreach`) y, desde PHP 8, la expresión `match`, más segura que `switch` porque compara de forma estricta y devuelve un valor.

Condicionales y match

```

1 <?php
2 $nota = 8;
3 if ($nota >= 7) {
4     echo 'Aprobado';
5 } elseif ($nota >= 5) {
6     echo 'Recuperacion';
7 } else {
8     echo 'Reprobado';
9 }
10
11 // match (PHP 8): comparacion estricta y devuelve valor
12 $letra = match (true) {
13     $nota >= 9 => 'A',
14     $nota >= 7 => 'B',
15     default    => 'C',
16 };
17 echo $letra;
```

Bucles

```

1 <?php
2 for ($i = 1; $i <= 3; $i++) {
3     echo $i;           // 123
4 }
5
6 $colores = ['rojo', 'verde', 'azul'];
7 foreach ($colores as $c) {
8     echo $c . ' ';     // rojo verde azul
9 }
```

`foreach` es la forma idiomática de recorrer arrays en PHP y la que más usarás al mostrar datos de una base. Reserva `for` para cuando necesites el índice numérico de forma explícita.

3.6 Arrays

El array es la estructura de datos más versátil de PHP: sirve como lista (índices numéricos) y como diccionario (claves de texto). Puede anidarse para representar datos complejos, como una fila de una tabla o una lista de filas.

Arrays

```

1 <?php
2 // Indexado (lista)
3 $nums = [10, 20, 30];
4 echo $nums[0];           // 10
5
6 // Asociativo (clave => valor)
7 $persona = ['nombre' => 'Ana', 'edad' => 21];
8 echo $persona['nombre']; // Ana
9
10 // Multidimensional (lista de filas)
11 $estudiantes = [
12     ['nombre' => 'Ana', 'carrera' => 'Software'],
13     ['nombre' => 'Luis', 'carrera' => 'Telematica'],
14 ];
15 echo $estudiantes[1]['nombre']; // Luis
16
17 // Funciones utiles
18 $nums[] = 40;           // agrega al final
19 echo count($nums);      // 4
20 sort($nums);           // ordena

```

Familiarízate con las funciones de array (`count`, `sort`, `array_map`, `array_filter`, `in_array`, `array_keys`): resuelven en una línea tareas que de otro modo requerirían bucles, y son muy preguntadas en evaluaciones.

3.7 Funciones

Una función agrupa código reutilizable. En PHP 8 deberías declarar los tipos de los parámetros y del valor de retorno: el código se vuelve más claro y los errores se detectan antes. Existen también las funciones flecha, una sintaxis breve para funciones de una sola expresión.

Funciones tipadas y flecha

```

1 <?php
2 function sumar(int $a, int $b): int {
3     return $a + $b;
4 }
5 echo sumar(2, 3);           // 5
6
7 // Parametro con valor por defecto y tipo nullable
8 function saludar(string $nombre = 'invitado'): string {
9     return "Hola, $nombre";
10 }
11
12 // Funcion flecha (arrow function): captura variables del entorno
13 $factor = 10;
14 $multiplicar = fn(int $x): int => $x * $factor;
15 echo $multiplicar(5);       // 50

```

Una buena función hace una sola cosa y la hace bien. Si una función crece demasiado o necesita muchos comentarios para entenderse, suele ser señal de que conviene dividirla.

3.8 Tipado estricto

Por defecto, PHP convierte tipos de forma automática al llamar funciones. Si añades `declare(strict_types=1);` como primera instrucción del archivo, PHP exige que los tipos coincidan exactamente, lo que evita conversiones silenciosas peligrosas. Es una práctica profesional muy recomendable.

Tipado estricto

```

1 <?php
2 declare(strict_types=1);    // primera línea del archivo
3
4 function area(float $base, float $altura): float {
5     return ($base * $altura) / 2;
6 }
7 echo area(3, 4);           // 6 (int aceptado como float)
8 // area('3', 4);           // ERROR: '3' es string, no float

```

3.9 Ejercicios — nivel básico

Estos ejercicios consolidan la sintaxis que acabas de ver. Resuélvelos con el servidor embebido (`php -S`) o por terminal con `php archivo.php`. Intenta no mirar los ejemplos mientras los resuelves.

Pon en práctica los fundamentos:

1. **[Básico]** Escribe un script que muestre tu nombre y tu carrera en dos líneas usando `echo`.
2. **[Básico]** Declara dos variables numéricas y muestra su suma, resta, producto y división.
3. **[Básico]** Crea una constante `IVA = 0.15`, un precio, y muestra el precio final con IVA incluido.

4. **[Básico]** Demuestra la diferencia entre comillas simples y dobles imprimiendo una variable de las dos formas.
5. **[Básico]** Dado un array de cinco notas, calcula y muestra el promedio (usa `array_sum` y `count`).
6. **[Básico]** Escribe una función `esPar(int $n): bool` y pruébala con varios valores.
7. **[Básico]** Usa `match` para convertir una nota numérica (0–10) en una letra (A, B, C, D, F).
8. **[Básico]** Recorre con `foreach` un array asociativo de país => capital y muestra cada par.

Solución del ejercicio 5 (promedio)

Una posible solución:

Solución 3.9.5

```
1 <?php
2 $notas = [7, 8.5, 6, 9, 10];
3 $promedio = array_sum($notas) / count($notas);
4 echo 'Promedio: ' . round($promedio, 2);
```

4 PHP del lado del servidor: la web dinámica

Ahora aplicarás PHP a su terreno natural: recibir datos del usuario y responder con HTML generado dinámicamente. Aquí aparecen los formularios, las superglobales y, muy importante, las dos reglas de oro de la seguridad: validar la entrada y escapar la salida.

4.1 El ciclo de una petición

Cuando el usuario envía un formulario o abre una URL, el navegador manda una petición HTTP a tu script PHP. El script lee los datos de la petición, hace su trabajo (validar, consultar la base, calcular) y devuelve una respuesta HTML. Comprender este ciclo es la clave de todo el desarrollo web.

4.2 Las superglobales

PHP pone los datos de la petición a tu disposición en unos arrays especiales llamados superglobales, accesibles desde cualquier parte del código. Los más importantes para empezar son los que contienen los datos enviados por el usuario y la información del servidor.

Superglobal	Contenido
<code>\$_GET</code>	Datos enviados por la URL (cadena de consulta <code>?clave=valor</code>).
<code>\$_POST</code>	Datos enviados en el cuerpo de un formulario (<code>method=post</code>).

Superglobal	Contenido
<code>\$_REQUEST</code>	Combina GET, POST y COOKIE (usar con cautela).
<code>\$_SERVER</code>	Información del servidor y la petición (método, ruta...).
<code>\$_SESSION</code>	Datos de la sesión del usuario (ver sección 7).
<code>\$_FILES</code>	Archivos subidos mediante un formulario.

4.3 Procesamiento de formularios (GET vs POST)

Un formulario HTML envía sus datos a un script PHP mediante GET o POST. Usa GET para consultas que se pueden repetir y compartir por URL (como una búsqueda); usa POST para acciones que modifican datos o que no deben quedar en la URL (como un registro o un inicio de sesión).

Formulario y procesamiento (POST)

```

1 <!-- formulario.php : el formulario -->
2 <form method="post" action="procesar.php">
3   <input type="text" name="nombre" required>
4   <button type="submit">Enviar</button>
5 </form>
6
7 <?php // procesar.php : lee y responde
8 $nombre = $_POST['nombre'] ?? ''; // ?? evita aviso si no existe
9 echo 'Recibido: ' . htmlspecialchars($nombre);

```

Observa el operador `??`: si la clave no existe (por ejemplo, si se accede directamente a `procesar.php`), devuelve una cadena vacía en lugar de provocar un aviso. Y observa `htmlspecialchars`: nunca imprimas datos del usuario sin escaparlos.

4.4 Validación y saneamiento de la entrada

Nunca confíes en los datos que llegan del navegador: pueden venir vacíos, mal formados o ser maliciosos. Valida (comprueba que cumplen las reglas) y sanea (elimina lo que sobra) en el servidor. PHP ofrece `filter_var` con filtros listos para los casos más comunes.

Validar con `filter_var`

```

1 <?php
2 declare(strict_types=1);
3
4 $correo = trim($_POST['correo'] ?? '');
5 $edad   = $_POST['edad'] ?? '';
6
7 $errores = [];
8 if ($correo === '') {
9     $errores[] = 'El correo es obligatorio.';
10 } elseif (!filter_var($correo, FILTER_VALIDATE_EMAIL)) {
11     $errores[] = 'El correo no tiene un formato valido.';
12 }
13 if (filter_var($edad, FILTER_VALIDATE_INT) === false) {
14     $errores[] = 'La edad debe ser un numero entero.';
15 }
16
17 if ($errores) {
18     foreach ($errores as $e) echo htmlspecialchars($e) . '<br>';
19 } else {
20     echo 'Datos validos.';
21 }

```

Regla de oro 1: validar la entrada

La validación del navegador (atributos como **required**) es solo comodidad para el usuario: se puede saltar con las herramientas de desarrollo. La validación que cuenta para la seguridad es SIEMPRE la del servidor.

4.5 Escapado de la salida (anti-XSS)

El reverso de validar la entrada es escapar la salida. Si imprimes datos del usuario directamente en el HTML, un atacante podría inyectar etiquetas `<script>` y ejecutar código en el navegador de otras personas: es el ataque XSS (Cross-Site Scripting). La defensa es convertir los caracteres especiales en entidades con `htmlspecialchars`.

Escapar siempre la salida

```

1 <?php
2 $comentario = $_POST['comentario'] ?? '';
3
4 // MAL: vulnerable a XSS
5 // echo $comentario;
6
7 // BIEN: el navegador muestra el texto, no lo ejecuta
8 echo htmlspecialchars($comentario, ENT_QUOTES, 'UTF-8');

```

Regla de oro 2: escapar la salida

Aplica `htmlspecialchars` a TODO dato que provenga del usuario o de la base de datos antes de mostrarlo en HTML. Es la defensa estándar contra XSS.

4.6 Generar HTML5 semántico desde PHP

PHP intercala lógica y HTML con naturalidad. Para que el resultado sea profesional, genera HTML5 semántico (con `header`, `nav`, `main`, `article`, `footer`) y separa, en lo posible, el cálculo de la presentación. Una técnica limpia es preparar los datos arriba y «pintar» abajo.

Datos arriba — presentación abajo

```

1 <?php
2 $titulo = 'Bienvenida';
3 $items = ['Inicio', 'Servicios', 'Contacto'];
4 ?>
5 <!DOCTYPE html>
6 <html lang="es">
7 <head><meta charset="UTF-8"><title><?= htmlspecialchars($titulo) ?></title></head>
8 <body>
9   <header><h1><?= htmlspecialchars($titulo) ?></h1></header>
10  <nav><ul>
11    <?php foreach ($items as $it): ?>
12      <li><?= htmlspecialchars($it) ?></li>
13    <?php endforeach; ?>
14  </ul></nav>
15  <main><p>Contenido principal.</p></main>
16 </body>
17 </html>

```

El atajo `<?= ... ?>` equivale a `<?php echo ... ?>` y es ideal en plantillas. La sintaxis alternativa de los bucles (`foreach (...): ... endforeach;`) hace el HTML más legible cuando se mezcla con etiquetas.

4.7 Ejercicios — nivel intermedio

Estos ejercicios te acercan a problemas reales de la web. Aplica en todos las dos reglas de oro: valida en el servidor y escapa al mostrar.

Practica el lado del servidor:

1. **[Intermedio]** Crea un formulario que pida nombre y correo, y un script que valide ambos con `filter_var` y muestre los errores o un mensaje de éxito.
2. **[Intermedio]** Haz una «calculadora» web: un formulario con dos números y una operación (select), y un script que muestre el resultado usando `match`.
3. **[Intermedio]** Construye una página que reciba `?nombre=` por GET y salude; si no llega, saluda a «invitado». Escapa la salida.

4. **[Intermedio]** Muestra una tabla HTML a partir de un array multidimensional de estudiantes, escapando cada celda.
5. **[Intermedio]** Implementa el patrón Post-Redirect-Get: tras procesar un POST válido, redirige con `header('Location: ...')` para que recargar no reenvíe el formulario.
6. **[Intermedio]** Añade un contador de visitas que se guarde en un archivo de texto (`file_get_contents/file_put_contents`).

5 Organización del código

A medida que un proyecto crece, mantener todo en un archivo se vuelve inmanejable. En esta sección verás cómo dividir el código en archivos, organizarlo con espacios de nombres, gestionar dependencias con Composer y manejar errores de forma profesional.

5.1 Incluir archivos: `include` y `require`

Puedes dividir el código y reutilizarlo incluyendo unos archivos en otros. `include` y `require` insertan el contenido de otro archivo; la diferencia es que `require` detiene el programa si el archivo no existe (úsalo para dependencias imprescindibles), mientras que `include` solo emite un aviso.

include / require

```
1 <?php
2 // config.php define la conexion; es imprescindible:
3 require __DIR__ . '/config/db.php';
4
5 // una cabecera reutilizable; si falta, no es critico:
6 include __DIR__ . '/vistas/cabecera.php';
7
8 // require_once evita incluir dos veces el mismo archivo:
9 require_once __DIR__ . '/src/utiles.php';
```

La constante mágica `__DIR__` contiene la ruta de la carpeta del archivo actual; úsala para construir rutas absolutas y evitar problemas según desde dónde se ejecute el script.

5.2 Espacios de nombres (namespaces)

Cuando un proyecto tiene muchas clases, los nombres pueden chocar. Los espacios de nombres organizan el código en «carpetas lógicas» y evitan colisiones, igual que los paquetes en Java. Se declaran al inicio del archivo y se importan con `use`.

Namespaces

```

1 <?php
2 // src/Modelo/Estudiante.php
3 namespace App\Modelo;
4
5 class Estudiante {
6     public function __construct(
7         public string $nombre,
8         public string $correo,
9     ) {}
10 }
11
12 // index.php : importar y usar
13 use App\Modelo\Estudiante;
14 $e = new Estudiante('Ana', 'ana@uteq.edu.ec');
```

5.3 Composer y el autoloader PSR-4

Composer es el gestor de dependencias de PHP (equivale a npm en Node o a Maven en Java). Además de instalar librerías de terceros, genera un «autoloader» que carga tus clases automáticamente al usarlas, siguiendo el estándar PSR-4, sin necesidad de **require** por cada clase.

composer.json (autoload PSR-4)

```

1 {
2     "name": "uteq/practica-php",
3     "require": {
4         "php": ">=8.3"
5     },
6     "autoload": {
7         "psr-4": { "App\\": "src/" }
8     }
9 }
```

Tras definirlo, ejecutas **composer dump-autoload** y, en tu **index.php**, basta con **require 'vendor/autoload.php'**; para que todas las clases de **src/** (bajo el namespace **App**) se carguen solas.

5.4 Manejo de errores y excepciones

Los programas fallan: un archivo no existe, la base de datos no responde, un dato es inválido. PHP moderno gestiona estos casos con excepciones, que se lanzan con **throw** y se capturan con **try/catch**. Así separas el flujo normal del manejo de errores y evitas que el programa se caiga de forma descontrolada.

try / catch

```

1 <?php
2 function dividir(float $a, float $b): float {
3     if ($b === 0.0) {
4         throw new InvalidArgumentException('No se puede dividir por cero');
5     }
6     return $a / $b;
7 }
8
9 try {
10     echo dividir(10, 0);
11 } catch (InvalidArgumentException $e) {
12     echo 'Error: ' . $e->getMessage();
13 } finally {
14     echo ' (operacion finalizada)';
15 }

```

En desarrollo vs en producción

Durante el desarrollo conviene ver los errores en pantalla; en producción, NUNCA: muestra un mensaje genérico al usuario y registra el detalle en un archivo de log (`error_log`). Mostrar trazas de error filtra información útil a un atacante (OWASP A05).

5.5 Ejercicios — organización

Estos ejercicios te hacen estructurar el código como en un proyecto real.

Organiza y reutiliza:

1. **[Intermedio]** Separa una página en tres archivos: `cabecera.php`, `pie.php` y `index.php`, e inclúyelos con `require`.
2. **[Intermedio]** Crea una clase `Calculadora` en `src/` con namespace `App`, configúrala con Composer (PSR-4) y úsala desde `index.php`.
3. **[Avanzado]** Escribe una función que lea un archivo y lance una excepción si no existe; captúrala y muestra un mensaje amigable.
4. **[Avanzado]** Implementa un pequeño «logger» que escriba mensajes con fecha y hora en `app.log` usando excepciones para los fallos de escritura.

6 Programación orientada a objetos

La programación orientada a objetos (OOP) organiza el código en torno a «objetos» que combinan datos y comportamiento. Es la base de las aplicaciones grandes y de los frameworks. PHP 8 tiene un modelo de objetos moderno y conciso que verás a continuación.

6.1 Clases, propiedades y métodos

Una clase es un molde que define qué datos (propiedades) y qué acciones (métodos) tendrán sus objetos. De una clase se crean instancias con **new**. Dentro de los métodos, **\$this** se refiere al objeto actual.

Clase basica

```

1 <?php
2 declare(strict_types=1);
3
4 class Cuenta {
5     private float $saldo = 0.0;      // propiedad privada
6
7     public function depositar(float $monto): void {
8         $this->saldo += $monto;
9     }
10    public function getSaldo(): float {
11        return $this->saldo;
12    }
13 }
14
15 $c = new Cuenta();
16 $c->depositar(100);
17 echo $c->getSaldo();    // 100

```

La visibilidad (**public**, **private**, **protected**) controla quién puede acceder a cada miembro. Como norma, las propiedades se declaran **private** y se exponen solo lo necesario mediante métodos: es el principio de encapsulación.

6.2 Constructores y promoción de propiedades

El constructor (**__construct**) se ejecuta al crear el objeto y sirve para inicializarlo. PHP 8 introdujo la «promoción de propiedades en el constructor»: declarar y asignar las propiedades directamente en los parámetros, lo que reduce muchísimo el código repetitivo.

Constructor con promocion (PHP 8)

```

1 <?php
2 class Estudiante {
3     public function __construct(
4         public string $nombre,
5         public string $correo,
6         private string $carrera = 'Software',
7     ) {}
8
9     public function ficha(): string {
10        return "$this->nombre <$this->correo>";
11    }
12 }
13
14 $e = new Estudiante('Ana', 'ana@uteq.edu.ec');
15 echo $e->ficha();

```

6.3 Herencia, interfaces, clases abstractas y traits

La OOP permite reutilizar y organizar comportamiento de varias maneras. La herencia (**extends**) crea una clase a partir de otra; las interfaces (**interface/implements**) definen un contrato que las clases deben cumplir; las clases abstractas mezclan ambas ideas; y los traits permiten compartir métodos entre clases no relacionadas.

Interfaz y herencia

```

1 <?php
2 interface Figura {
3     public function area(): float;
4 }
5
6 class Circulo implements Figura {
7     public function __construct(private float $radio) {}
8     public function area(): float {
9         return 3.1416 * $this->radio ** 2;
10    }
11 }
12
13 class Cuadrado implements Figura {
14     public function __construct(private float $lado) {}
15     public function area(): float {
16         return $this->lado ** 2;
17    }
18 }
19
20 $figuras = [new Circulo(2), new Cuadrado(3)];
21 foreach ($figuras as $f) {
22     echo $f->area() . ' '; // polimorfismo
23 }
```

Este ejemplo muestra el polimorfismo: distintos objetos responden al mismo método **area()** cada uno a su manera. Programar contra interfaces (y no contra clases concretas) es una de las claves del software flexible.

6.4 Enumeraciones (enums)

Desde PHP 8.1, las enumeraciones representan un conjunto fijo de valores posibles, como los estados de un pedido o los roles de un usuario. Sustituyen con ventaja a las constantes sueltas, porque el tipo garantiza que solo se usen valores válidos.

Enum (PHP 8.1)

```

1 <?php
2 enum Estado: string {
3     case Activo    = 'activo';
4     case Inactivo  = 'inactivo';
5     case Suspendido = 'suspendido';
6 }
7
8 function describir(Estado $e): string {
9     return match ($e) {
10         Estado::Activo    => 'La cuenta esta activa',
11         Estado::Inactivo  => 'La cuenta esta inactiva',
12         Estado::Suspendido => 'La cuenta esta suspendida',
13     };
14 }
15 echo describir(Estado::Activo);

```

6.5 Inyección de dependencias (concepto)

Un objeto suele necesitar otros para trabajar (por ejemplo, un repositorio necesita una conexión a la base de datos). La inyección de dependencias consiste en pasarle esas colaboraciones desde fuera (normalmente por el constructor) en lugar de crearlas dentro. Así el código es más flexible y mucho más fácil de probar.

Inyeccion por constructor

```

1 <?php
2 class RepositorioEstudiantes {
3     public function __construct(private PDO $pdo) {} // se inyecta
4
5     public function todos(): array {
6         return $this->pdo->query('SELECT * FROM estudiantes')
7             ->fetchAll();
8     }
9 }
10 // El que crea el repositorio decide que PDO usar:
11 // $repo = new RepositorioEstudiantes($pdo);

```

6.6 Ejercicios — OOP

Estos ejercicios te hacen diseñar con objetos, no solo escribir funciones sueltas.

Modela con clases:

1. **[Intermedio]** Crea una clase `Producto` con nombre y precio (promoción de propiedades) y un método que devuelva el precio con IVA.
2. **[Intermedio]** Define una interfaz `Notificable` con un método `enviar(string $msg)` e implémentala en `NotificacionEmail` y `NotificacionSMS`.

3. **[Avanzado]** Modela una jerarquía **Figura** (abstracta) con **Circulo**, **Rectangulo** y **Triangulo**, cada una con su `area()`; recórrelas con un bucle.
4. **[Avanzado]** Crea un enum **Rol** (**Admin**, **Docente**, **Estudiante**) y una función que devuelva los permisos de cada rol con `match`.
5. **[Avanzado]** Refactoriza el ejercicio de la calculadora para que use una clase **Calculadora** con inyección de la operación.

7 Sesiones, cookies y estado

Como HTTP no recuerda al usuario entre peticiones, necesitamos un mecanismo para mantener el estado: por ejemplo, para saber que alguien ya inició sesión. PHP resuelve esto con cookies y, sobre todo, con sesiones. En esta sección montarás también un inicio de sesión básico y seguro.

7.1 Cookies

Una cookie es un pequeño dato que el servidor pide al navegador que guarde y reenvíe en las siguientes peticiones. Sirve, por ejemplo, para recordar una preferencia. Se crean con `setcookie` antes de enviar cualquier salida y se leen desde la superglobal `$_COOKIE`.

Cookies

```
1 <?php
2 // Crear una cookie que dura 7 dias:
3 setcookie('tema', 'oscuro', time() + 7*24*3600, '/');
4
5 // Leerla (estara disponible en la SIGUIENTE peticion):
6 $tema = $_COOKIE['tema'] ?? 'claro';
7 echo "Tema actual: $tema";
```

7.2 Sesiones

La sesión es el mecanismo principal para mantener el estado. A diferencia de la cookie, los datos se guardan en el servidor y el navegador solo conserva un identificador. Se inicia con `session_start()` (antes de cualquier salida) y se accede mediante la superglobal `$_SESSION`.

Sesiones

```

1 <?php
2 session_start(); // siempre al inicio
3
4 // Guardar datos en la sesion:
5 $_SESSION['usuario'] = 'ana';
6 $_SESSION['rol'] = 'docente';
7
8 // Leerlos en cualquier pagina (tras session_start):
9 echo 'Hola, ' . htmlspecialchars($_SESSION['usuario']) ?? 'invitado';
10
11 // Cerrar sesion:
12 // session_unset(); session_destroy();

```

7.3 Autenticación básica: un inicio de sesión seguro

Con sesiones y el hash de contraseñas puedes implementar un inicio de sesión. La regla esencial es: las contraseñas NUNCA se guardan en texto plano. Se almacena un «hash» generado con `password_hash` (que usa BCrypt o Argon2) y se comprueba con `password_verify`.

Login con hash de contraseña

```

1 <?php
2 session_start();
3
4 // Al registrar (una vez): guardar el HASH, no la clave:
5 // $hash = password_hash($claveEnClaro, PASSWORD_DEFAULT);
6 $hashGuardado = password_hash('secreta123', PASSWORD_DEFAULT);
7
8 // Al iniciar sesion: comparar lo que escribe el usuario:
9 $claveIngresada = $_POST['clave'] ?? '';
10 if (password_verify($claveIngresada, $hashGuardado)) {
11     $_SESSION['autenticado'] = true;
12     echo 'Bienvenido';
13 } else {
14     echo 'Credenciales incorrectas';
15 }

```

Por qué `password_hash` y no `md5`

`password_hash` añade una «sal» aleatoria y es deliberadamente lento, lo que frena los ataques de fuerza bruta. Algoritmos como `md5` o `sha1` son rápidos y sin sal: NO sirven para contraseñas.

7.4 El patrón Post-Redirect-Get

Tras procesar un formulario enviado por POST, conviene responder con una redirección (`header('Location: ...')`) en lugar de mostrar la página directamente. Así, si el usuario recarga, no se reenvía el formulario (evitando, por ejemplo, un doble registro). Es el patrón Post-Redirect-Get (PRG).

Patron PRG

```
1 <?php
2 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
3     // ... procesar e insertar ...
4     header('Location: lista.php'); // redirige (302)
5     exit;                          // importante: detener aquí
6 }
```

7.5 Ejercicios — sesiones y estado

Estos ejercicios integran estado y seguridad básica.

Mantén el estado:

1. **[Intermedio]** Guarda en sesión el nombre que el usuario escribe en un formulario y muéstralo en otra página.
2. **[Intermedio]** Implementa un selector de tema (claro/oscuro) que recuerde la elección con una cookie.
3. **[Avanzado]** Crea un login con dos páginas (`login.php` y `panel.php`) usando `password_hash/` `password_verify` y protege el panel para que solo entren usuarios autenticados.
4. **[Avanzado]** Añade un «cerrar sesión» que destruya la sesión y redirija al login.

8 Acceso a base de datos con PDO

Las aplicaciones reales guardan y consultan datos en una base de datos. PHP accede a ellas mediante PDO (PHP Data Objects), una capa uniforme que funciona con PostgreSQL, MySQL y otros motores cambiando solo la cadena de conexión. Aquí aprenderás a conectar, consultar y, sobre todo, a hacerlo de forma segura.

8.1 ¿Por qué PDO?

PDO ofrece una interfaz común para distintos motores y, lo más importante, soporta sentencias preparadas, que son la defensa contra la inyección SQL. Frente a las antiguas funciones específicas de cada base, PDO es portable, moderno y seguro; es la opción recomendada hoy.

8.2 La conexión

Para conectar necesitas un DSN (Data Source Name) que describe el motor, el host, el puerto y la base, más el usuario y la contraseña. Conviene configurar PDO para que lance excepciones ante los errores y para que devuelva las filas como arrays asociativos. Usaremos la misma base PostgreSQL de la guía de Perl.

Conexion con PDO (config/db.php)

```

1 <?php
2 declare(strict_types=1);
3
4 $dsn = 'pgsql:host=127.0.0.1;port=5432;dbname=appweb2026ppa';
5 $usuario = 'postgres';
6 $clave = 'P@7Gr3$QL'; // comillas SIMPLES por el $ y la @
7
8 try {
9     $pdo = new PDO($dsn, $usuario, $clave, [
10         PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,
11         PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,
12         PDO::ATTR_EMULATE_PREPARES => false,
13     ]);
14 } catch (PDOException $e) {
15     // En produccion: registrar y mostrar mensaje generico
16     die('Error de conexion: ' . $e->getMessage());
17 }

```

Para MySQL en vez de PostgreSQL

Solo cambia el DSN: `mysql:host=127.0.0.1;port=3306;dbname=appweb2026ppa;charset=utf8mb4`. El resto del código PDO es idéntico: esa portabilidad es una de las ventajas de PDO.

8.3 Sentencias preparadas (anti-SQL injection)

Nunca incrustes datos del usuario directamente en una cadena SQL: es la puerta a la inyección SQL. En su lugar, usa sentencias preparadas con marcadores ? (o con nombre :clave) y pasa los valores aparte: el motor los trata como datos, nunca como código.

Sentencias preparadas

```

1 <?php
2 // MAL (vulnerable): NUNCA hagas esto
3 // $pdo->query("SELECT * FROM estudiantes WHERE correo = '$correo'");
4
5 // BIEN: marcadores y valores separados
6 $stmt = $pdo->prepare(
7     'SELECT * FROM estudiantes WHERE correo = ?'
8 );
9 $stmt->execute([$correo]);
10 $fila = $stmt->fetch(); // una fila (o false)
11
12 // Con marcadores con nombre:
13 $stmt = $pdo->prepare(
14     'INSERT INTO estudiantes (nombre, correo, carrera)
15     VALUES (:n, :c, :ca)'
16 );
17 $stmt->execute([':n' => $nombre, ':c' => $correo, ':ca' => $carrera]);

```

8.4 CRUD completo

CRUD son las cuatro operaciones básicas sobre datos: crear, leer, actualizar y eliminar. Con PDO y sentencias preparadas, cada una es directa. Observa cómo se recuperan los resultados de una consulta y cómo se obtiene el número de filas afectadas.

Las cuatro operaciones

```

1 <?php
2 // CREATE
3 $pdo->prepare('INSERT INTO estudiantes (nombre, correo, carrera) VALUES (?, ?, ?)')
4     ->execute([$nombre, $correo, $carrera]);
5
6 // READ (todas las filas)
7 $filas = $pdo->query('SELECT * FROM estudiantes ORDER BY id')
8     ->fetchAll();
9 foreach ($filas as $f) {
10     echo htmlspecialchars($f['nombre']) . '<br>';
11 }
12
13 // UPDATE
14 $stmt = $pdo->prepare('UPDATE estudiantes SET correo = ? WHERE id = ?');
15 $stmt->execute([$correo, $id]);
16 echo $stmt->rowCount() . ' fila(s) actualizada(s)';
17
18 // DELETE
19 $pdo->prepare('DELETE FROM estudiantes WHERE id = ?')->execute([$id]);

```

8.5 Transacciones

Cuando varias operaciones deben tener éxito o fracasar como un bloque (por ejemplo, transferir saldo entre dos cuentas), se agrupan en una transacción. Si algo falla, se deshace todo con un `rollback`; si todo va bien, se confirma con `commit`.

Transaccion

```

1 <?php
2 try {
3     $pdo->beginTransaction();
4     $pdo->prepare('UPDATE cuentas SET saldo = saldo - ? WHERE id = ?')
5         ->execute([100, 1]);
6     $pdo->prepare('UPDATE cuentas SET saldo = saldo + ? WHERE id = ?')
7         ->execute([100, 2]);
8     $pdo->commit(); // todo correcto
9 } catch (PDOException $e) {
10     $pdo->rollback(); // deshacer todo
11     echo 'Transaccion cancelada';
12 }

```

8.6 Ejercicios — base de datos

Estos ejercicios requieren PostgreSQL en marcha y la tabla **estudiantes**. Aplica siempre sentencias preparadas.

Trabaja con datos:

1. **[Intermedio]** Conéctate con PDO y muestra todos los estudiantes en una tabla HTML, escapando cada celda.
2. **[Intermedio]** Inserta un estudiante con una sentencia preparada y muestra el id generado.
3. **[Avanzado]** Implementa una búsqueda por nombre con LIKE y marcadores, mostrando los resultados.
4. **[Avanzado]** Escribe una transacción que inserte una carrera y varios estudiantes asociados; si uno falla, que no quede nada.
5. **[Avanzado]** Crea un reporte que cuente estudiantes por carrera con GROUP BY.

9 Proyecto integrador: CRUD de estudiantes

Es hora de unir todo en una aplicación completa: un registro de estudiantes con alta, listado (reporte) y eliminación, conectado a PostgreSQL con PDO, con validación, escapado y el patrón PRG. Este proyecto resume el corazón de la unidad.

9.1 Estructura del proyecto

Separaremos la conexión (en `config/db.php`) del punto de entrada (`index.php`). En un proyecto mayor añadiríamos clases y vistas, pero para la guía mantenemos dos archivos claros y comentados.

Estructura

```
1 crud_php/
2 |- config/
3 |   |- db.php           # conexion PDO
4 |   |- index.php        # listado + formulario + acciones
5 |   |- schema.sql       # tabla estudiantes
```

9.2 El esquema de la base

La tabla es la misma de la guía de Perl, de modo que puedes reutilizar la base **appweb2026ppa**. Ejecuta este SQL en psql o pgAdmin (o deja que lo cree tu propio script).

schema.sql

```
1 CREATE TABLE IF NOT EXISTS estudiantes (  
2     id          SERIAL          PRIMARY KEY,  
3     nombre     VARCHAR(100) NOT NULL,  
4     correo     VARCHAR(120) NOT NULL,  
5     carrera    VARCHAR(80)  NOT NULL,  
6     creado     TIMESTAMP       NOT NULL DEFAULT NOW()  
7 );
```

9.3 La conexión

Reutilizamos la conexión PDO de la sección 8, en su propio archivo, para incluirla donde haga falta. Tener la conexión en un solo lugar evita repetir credenciales y facilita los cambios.

config/db.php

```
1 <?php  
2 declare(strict_types=1);  
3 $dsn = 'pgsql:host=127.0.0.1;port=5432;dbname=appweb2026ppa';  
4 try {  
5     $pdo = new PDO($dsn, 'postgres', 'P@7Gr3$QL', [  
6         PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,  
7         PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,  
8     ]);  
9 } catch (PDOException $e) {  
10     die('Error de conexion');  
11 }
```

9.4 El punto de entrada: lógica

El archivo `index.php` hace tres cosas: si llega un POST, da de alta y redirige (PRG); si llega `?eliminar=ID`, borra y redirige; en cualquier otro caso, muestra el formulario y el listado. Primero, la parte de lógica, arriba del HTML.

index.php (lógica)

```

1 <?php
2 declare(strict_types=1);
3 require __DIR__ . '/config/db.php';
4
5 // ALTA (POST) -> insertar y redirigir (PRG)
6 if ($_SERVER['REQUEST_METHOD'] === 'POST') {
7     $nombre = trim($_POST['nombre'] ?? '');
8     $correo = trim($_POST['correo'] ?? '');
9     $carrera = trim($_POST['carrera'] ?? '');
10    if ($nombre !== '' && filter_var($correo, FILTER_VALIDATE_EMAIL) && $carrera !== '') {
11        $pdo->prepare(
12            'INSERT INTO estudiantes (nombre, correo, carrera) VALUES (?, ?, ?)'
13        )->execute([$nombre, $correo, $carrera]);
14    }
15    header('Location: index.php');
16    exit;
17 }
18
19 // ELIMINAR (GET ?eliminar=ID) -> borrar y redirigir
20 if (isset($_GET['eliminar'])) {
21     $id = (int) $_GET['eliminar'];
22     $pdo->prepare('DELETE FROM estudiantes WHERE id = ?')->execute([$id]);
23     header('Location: index.php');
24     exit;
25 }
26
27 // LISTAR (reporte)
28 $estudiantes = $pdo->query(
29     'SELECT id, nombre, correo, carrera FROM estudiantes ORDER BY id'
30 )->fetchAll();

```

9.5 El punto de entrada: presentación

Debajo de la lógica va el HTML5, que «pinta» el formulario de alta y la tabla de estudiantes. Cada dato del usuario se muestra con `htmlspecialchars` para evitar XSS. Observa la sintaxis alternativa del `foreach` para mantener el HTML legible.

index.php (HTML)

```

1 <!DOCTYPE html>
2 <html lang="es">
3 <head><meta charset="UTF-8"><title>Estudiantes</title></head>
4 <body>
5   <h1>Registro de estudiantes</h1>
6
7   <form method="post" action="index.php">
8     <input type="text" name="nombre" placeholder="Nombre" required>
9     <input type="email" name="correo" placeholder="Correo" required>
10    <input type="text" name="carrera" placeholder="Carrera" required>
11    <button type="submit">Guardar</button>
12  </form>
13
14  <table border="1" cellpadding="6">
15    <tr><th>ID</th><th>Nombre</th><th>Correo</th><th>Carrera</th><th></th></tr>
16    <?php foreach ($estudiantes as $e): ?>
17      <tr>
18        <td><?= (int) $e['id'] ?></td>
19        <td><?= htmlspecialchars($e['nombre']) ?></td>
20        <td><?= htmlspecialchars($e['correo']) ?></td>
21        <td><?= htmlspecialchars($e['carrera']) ?></td>
22        <td><a href="?eliminar=<?= (int) $e['id'] ?>">Eliminar</a></td>
23      </tr>
24    <?php endforeach; ?>
25  </table>
26 </body>
27 </html>

```

Con estos dos bloques en `index.php` tienes una aplicación CRUD funcional: ejecútala con Apache (en `htdocs`) o con `php -S localhost:8000` y ábrela en el navegador.

9.6 Ejercicio integrador

Amplía el proyecto para consolidar todo lo aprendido en la unidad.

Lleva el proyecto más lejos:

1. **[Avanzado]** Añade la operación **Editar**: un formulario que cargue los datos de un estudiante y los actualice con `UPDATE`.
2. **[Avanzado]** Agrega un reporte de «estudiantes por carrera» con `GROUP BY` debajo de la tabla.
3. **[Avanzado]** Protege la aplicación con un login (sección 7): solo usuarios autenticados pueden dar de alta o eliminar.
4. **[Avanzado]** Añade un token CSRF al formulario (sección 10) para proteger las acciones de alta y borrado.
5. **[Avanzado]** Refactoriza el acceso a datos a una clase `RepositorioEstudiantes` con inyección de PDO.

10 Seguridad en aplicaciones PHP

La seguridad no es un añadido final, sino una forma de programar desde la primera línea. Esta sección reúne las defensas esenciales; varias ya aparecieron a lo largo de la guía, aquí se consolidan y se añaden las que faltan, en línea con el OWASP Top 10.

10.1 Validar la entrada y escapar la salida

Son las dos reglas de oro, y se repiten porque son la base de casi todo lo demás. Valida en el servidor todo lo que llega del usuario, y escapa con `htmlspecialchars` todo lo que muestras. Estas dos prácticas, por sí solas, eliminan una gran proporción de vulnerabilidades.

10.2 Inyección SQL

La inyección SQL ocurre cuando datos del usuario se incrustan en una consulta y alteran su significado. La defensa definitiva, como viste, son las sentencias preparadas de PDO con marcadores. Nunca construyas SQL concatenando variables del usuario, por inofensivas que parezcan.

10.3 Cross-Site Scripting (XSS)

El XSS inyecta scripts en páginas que verán otros usuarios. Se previene escapando la salida con `htmlspecialchars` (y `ENT_QUOTES` cuando el dato va dentro de un atributo). Para contenido enriquecido de terceros, se usan librerías de saneamiento de HTML, nunca la confianza ciega.

10.4 Falsificación de petición entre sitios (CSRF)

El CSRF engaña al navegador de un usuario autenticado para que realice una acción sin su consentimiento. La defensa habitual es incluir en cada formulario un «token» secreto e impredecible que el servidor genera y verifica; si no coincide, la acción se rechaza.

Token CSRF

```

1 <?php
2 session_start();
3 // Generar el token una vez y guardarlo en la sesion:
4 if (empty($_SESSION['csrf'])) {
5     $_SESSION['csrf'] = bin2hex(random_bytes(32));
6 }
7 ?>
8 <form method="post" action="accion.php">
9     <input type="hidden" name="csrf" value="<?= $_SESSION['csrf'] ?>">
10     <!-- ... resto del formulario ... -->
11 </form>
12
13 <?php // En accion.php, antes de procesar:
14 if (!hash_equals($_SESSION['csrf'] ?? '', $_POST['csrf'] ?? '')) {
15     http_response_code(403);
16     exit('Token CSRF invalido');
17 }

```

10.5 Gestión de contraseñas

Las contraseñas se guardan siempre como hash con `password_hash` (BCrypt o Argon2), nunca en texto plano ni con algoritmos rápidos como md5. Al verificar, se usa `password_verify`. Conviene además limitar los intentos de inicio de sesión para frenar ataques de fuerza bruta.

10.6 Configuración y cabeceras seguras

Una parte de la seguridad vive en la configuración. En producción, desactiva la muestra de errores (`display_errors = Off`) y regístralos en un log; sirve la web por HTTPS; y añade cabeceras de seguridad como `Content-Security-Policy` y `X-Frame-Options`. No dejes credenciales ni secretos dentro del repositorio.

Resumen de seguridad

Valida la entrada, escapa la salida, usa sentencias preparadas, protege los formularios con tokens CSRF, guarda las contraseñas con `password_hash`, sirve por HTTPS y mantén las dependencias actualizadas con Composer.

10.7 Ejercicios — seguridad

Estos ejercicios endurecen tus aplicaciones frente a ataques reales.

Asegura tu código:

1. **[Intermedio]** Toma un script vulnerable que concatena SQL y reescríbelo con sentencias preparadas.
2. **[Intermedio]** Detecta y corrige un XSS: una página que imprime un comentario sin escapar.

3. **[Avanzado]** Añade protección CSRF al CRUD del proyecto integrador.
4. **[Avanzado]** Implementa un límite de tres intentos de login por sesión, con bloqueo temporal.

11 Buenas prácticas y ecosistema moderno

Más allá de la sintaxis, escribir PHP profesional implica seguir convenciones y apoyarse en herramientas. Esta sección te orienta sobre los estándares, el ecosistema de paquetes, los frameworks y las pruebas, para que tu siguiente paso sea sólido.

11.1 Estándares PSR

Los PSR (PHP Standards Recommendations) son convenciones acordadas por la comunidad: PSR-12 para el estilo de código, PSR-4 para el autoloader de clases, y otros. Seguirlos hace que tu código sea coherente con el del resto del mundo PHP y más fácil de mantener en equipo.

11.2 Composer y Packagist

Composer gestiona las dependencias y Packagist es su repositorio público de paquetes. Casi cualquier necesidad (enrutamiento, plantillas, validación, envío de correo) tiene un paquete maduro que puedes añadir con un comando, en lugar de reinventarlo.

11.3 Frameworks: Laravel y Symfony

Para aplicaciones grandes, los frameworks aportan estructura y resuelven lo repetitivo. Laravel destaca por su productividad y curva amable; Symfony, por su robustez y componentes reutilizables. Conviene dominar primero PHP «a mano» —como en esta guía— para entender qué hace el framework por debajo. Por su importancia, los abordamos en detalle y comparados en la sección siguiente.

11.4 Pruebas con PHPUnit

Las pruebas automatizadas verifican que tu código hace lo que debe y que sigue funcionando tras los cambios. PHPUnit es el estándar en PHP. Empieza por probar tus funciones y clases puras (como una **Calculadora**), que son las más fáciles de testear.

Prueba con PHPUnit (idea)

```
1 <?php
2 use PHPUnit\Framework\TestCase;
3
4 final class CalculadoraTest extends TestCase {
5     public function testSuma(): void {
6         $c = new Calculadora();
7         $this->assertSame(5, $c->sumar(2, 3));
8     }
9 }
```

11.5 Calidad de código

Herramientas como PHP_CodeSniffer (estilo), PHPStan o Psalm (análisis estático) detectan errores y malos olores antes de ejecutar el programa. Integrarlas en tu flujo de trabajo eleva mucho la calidad con poco esfuerzo, y son habituales en entornos profesionales.

12 Frameworks de PHP: Laravel y Symfony comparados

A lo largo de la guía has construido todo a mano: el enrutamiento, la lectura de formularios, el escapado, el acceso a datos con PDO. Eso es justo lo que un framework automatiza. Ahora que entiendes los fundamentos, esta sección te muestra los dos frameworks más usados de PHP resolviendo las mismas tareas, para que veas el patrón en lugar de memorizar una herramienta concreta.

La idea no es que domines aquí Laravel o Symfony —eso requiere su propio curso—, sino que reconozcas qué aporta un framework y cómo se traducen a él los conceptos que ya conoces. Por eso iremos comparando, tarea por tarea, cómo se hace lo mismo en cada uno.

12.1 ¿Qué resuelve un framework y por qué comparar dos?

Un framework es un conjunto de herramientas y convenciones que te dan resueltos los problemas comunes de toda aplicación web, para que te concentres en la lógica de tu negocio. En lugar de reescribir el enrutamiento o el acceso a datos en cada proyecto, los usas ya hechos y probados.

Comparar Laravel y Symfony tiene sentido porque son los dos dominantes del ecosistema y porque, al ver la misma tarea en ambos, distingues lo esencial (el concepto) de lo accidental (la sintaxis de cada uno). De hecho, Laravel se apoya internamente en muchos componentes de Symfony, así que no son rivales tan distintos como parecen.

Esto es lo que cualquier framework moderno te da resuelto:

- **Enrutamiento:** asociar una URL a un trozo de código (un controlador).
- **MVC:** separar el modelo (datos), la vista (presentación) y el controlador (coordinación).
- **ORM:** trabajar con la base de datos mediante objetos, sin escribir SQL a mano.

- **Plantillas:** generar HTML de forma limpia y, muy importante, **escapado por defecto** (anti-XSS).
- **Validación, sesiones, seguridad (CSRF, hash de contraseñas) y consola** de comandos, todo integrado.

12.2 Los dos protagonistas: perfiles

Antes de comparar tareas, conviene un retrato rápido de cada uno para entender su filosofía. No hay uno «mejor»: hay uno más adecuado según el proyecto y el equipo.

Laravel prioriza la productividad y la comodidad del desarrollador. Sigue el principio de «convención sobre configuración» y trae «las baterías incluidas»: ORM (Eloquent), plantillas (Blade), autenticación, colas, etc. Su curva de aprendizaje es suave, por lo que es excelente para empezar y para construir productos con rapidez.

Symfony prioriza la robustez, la flexibilidad y la reutilización. Está hecho de componentes independientes y favorece la configuración explícita, lo que lo hace muy potente para sistemas grandes y de larga vida. Su curva es algo más pronunciada, pero enseña una arquitectura muy sólida y es un estándar en el mundo empresarial.

12.3 Comparativa 1: crear el proyecto

Ambos se instalan con Composer y traen un servidor de desarrollo para empezar en segundos. La diferencia está en que Laravel viene «completo» de fábrica, mientras que Symfony parte de un esqueleto al que añades lo que necesitas.

Laravel — crear el proyecto

```
1 # Requiere Composer instalado
2 composer create-project laravel/laravel mi-app
3 cd mi-app
4 php artisan serve           # http://localhost:8000
```

Symfony — crear el proyecto

```
1 # Requiere Composer (y, opcional, la CLI de Symfony)
2 composer create-project symfony/skeleton mi-app
3 cd mi-app
4 composer require webapp      # anade Twig, Doctrine, etc.
5 symfony serve                # http://localhost:8000
```

Observa el matiz: en Laravel todo está listo desde el inicio; en Symfony decides qué incorporar (aquí, el «stack web» con `composer require webapp`). Esa es, en miniatura, la diferencia de filosofía entre ambos.

12.4 Comparativa 2: una ruta y un controlador

El enrutamiento asocia una URL con un método de un controlador, que es el equivalente a lo que en el proyecto integrador hacías con `if ($_SERVER['REQUEST_METHOD'] ...)`. Laravel define las rutas en un archivo central; Symfony las declara como atributos sobre el propio método.

Laravel — ruta y controlador

```

1 <?php
2 // routes/web.php
3 use App\Http\Controllers\EstudianteController;
4 Route::get('/estudiantes', [EstudianteController::class, 'index']);
5
6 // app/Http/Controllers/EstudianteController.php
7 namespace App\Http\Controllers;
8 use App\Models\Estudiante;
9 class EstudianteController extends Controller {
10     public function index() {
11         $estudiantes = Estudiante::all();           // ORM Eloquent
12         return view('estudiantes.index', compact('estudiantes'));
13     }
14 }
```

Symfony — ruta y controlador

```

1 <?php
2 // src/Controller/EstudianteController.php
3 namespace App\Controller;
4 use App\Repository\EstudianteRepository;
5 use Symfony\Bundle\FrameworkBundle\Controller\AbstractController;
6 use Symfony\Component\HttpFoundation\Response;
7 use Symfony\Component\Routing\Attribute\Route;
8
9 class EstudianteController extends AbstractController {
10     #[Route('/estudiantes', name: 'estudiantes_index')]
11     public function index(EstudianteRepository $repo): Response {
12         return $this->render('estudiantes/index.html.twig', [
13             'estudiantes' => $repo->findAll(), // ORM Doctrine
14         ]);
15     }
16 }
```

En ambos, el resultado es el mismo: la URL `/estudiantes` ejecuta un método que obtiene los datos y los pasa a una vista. Fíjate en que Symfony «inyecta» el repositorio como argumento del método (inyección de dependencias, que viste en la sección de POO), mientras que Laravel usa el modelo directamente.

12.5 Comparativa 3: el modelo y el acceso a datos (ORM)

Aquí está una de las mayores diferencias de estilo. Ambos usan un ORM para no escribir SQL, pero con enfoques distintos: Laravel usa Eloquent, basado en el patrón Active Record (el propio objeto sabe guardarse); Symfony usa Doctrine, basado en Data Mapper (un gestor aparte se encarga de

persistir los objetos).

Laravel — modelo Eloquent (Active Record)

```

1 <?php
2 // app/Models/Estudiante.php (la tabla 'estudiantes' se infiere)
3 namespace App\Models;
4 use Illuminate\Database\Eloquent\Model;
5 class Estudiante extends Model {
6     protected $fillable = ['nombre', 'correo', 'carrera'];
7 }
8
9 // Uso: el propio objeto/clase opera sobre la base
10 Estudiante::all(); // listar
11 Estudiante::create(...); // crear
12 $e = Estudiante::find(1); // buscar por id
13 $e->update(['correo' => 'nuevo@uteq.edu.ec']);
14 $e->delete(); // eliminar

```

Symfony — entidad Doctrine (Data Mapper)

```

1 <?php
2 // src/Entity/Estudiante.php
3 namespace App\Entity;
4 use Doctrine\ORM\Mapping as ORM;
5 #[ORM\Entity]
6 class Estudiante {
7     #[ORM\Id] #[ORM\GeneratedValue]
8     #[ORM\Column] private int $id;
9     #[ORM\Column(length: 100)] private string $nombre;
10    // ... mas columnas, getters y setters ...
11 }
12
13 // Uso: un EntityManager ($em) persiste los objetos
14 $repo->findAll(); // listar
15 $em->persist($e); $em->flush(); // crear o actualizar
16 $em->remove($e); $em->flush(); // eliminar

```

¿Cuál es mejor? Depende. Active Record (Eloquent) es más directo y rápido de escribir; Data Mapper (Doctrine) separa mejor los objetos de la base, lo que escala mejor en sistemas complejos. Reconocerás aquí la misma idea de ORM que el JPA de tu Proyecto Fin de Curso en Java.

12.6 Comparativa 4: las plantillas (vistas)

Para generar HTML, cada framework trae su motor de plantillas: Blade en Laravel y Twig en Symfony. La gran ventaja frente al HTML incrustado que viste en el proyecto integrador es que **ambos escapan la salida por defecto**: la defensa anti-XSS deja de ser una tarea manual y pasa a ser automática.

Laravel — plantilla Blade

```

1  {{-- resources/views/estudiantes/index.blade.php --}}
2  <ul>
3  @foreach ($estudiantes as $e)
4      <li>{{ $e->nombre }} - {{ $e->correo }}</li>
5  @endforeach
6  </ul>
7  {{-- las llaves dobles {{ }} escapan automaticamente (anti-XSS) --}}
```

Symfony — plantilla Twig

```

1  {# templates/estudiantes/index.html.twig #}
2  <ul>
3  {% for e in estudiantes %}
4      <li>{{ e.nombre }} - {{ e.correo }}</li>
5  {% endfor %}
6  </ul>
7  {# las llaves dobles {{ }} tambien escapan por defecto #}
```

Las dos sintaxis son muy parecidas y mucho más limpias que mezclar `<?php ?>` con HTML. Recuerda el detalle clave: con Blade y Twig, mostrar un dato del usuario ya es seguro por defecto; solo desactivarías el escapado a propósito y con mucho cuidado.

12.7 Comparativa 5: la validación

Validar la entrada —otra de las reglas de oro— también está integrada. Laravel valida en el controlador con una lista de reglas; Symfony declara las reglas como atributos sobre las propiedades de la entidad, junto al modelo de datos.

Laravel — validacion en el controlador

```

1  <?php
2  $datos = $request->validate([
3      'nombre' => 'required|min:3',
4      'correo' => 'required|email',
5      'carrera' => 'required',
6  ]);
7  Estudiante::create($datos);    // solo se llega aqui si es valido
```

Symfony — validacion por atributos

```

1 <?php
2 use Symfony\Component\Validator\Constraints as Assert;
3
4 #[Assert\NotBlank]
5 #[Assert\Length(min: 3)]
6 private string $nombre;
7
8 #[Assert\NotBlank]
9 #[Assert\Email]
10 private string $correo;

```

En ambos casos, la validación que cuenta sigue ocurriendo en el servidor, igual que aprendiste; el framework solo te da una forma más declarativa y reutilizable de expresarla.

12.8 El CRUD de estudiantes, ahora con framework

Compara mentalmente con el proyecto integrador de la sección 9, donde escribiste la conexión, las consultas, el escapado y el enrutamiento a mano. Listar los estudiantes en un framework se reduce a esto (crear, actualizar y eliminar son igual de breves):

Laravel — listar estudiantes

```

1 <?php
2 // Ruta
3 Route::get('/estudiantes', [EstudianteController::class, 'index']);
4
5 // Controlador
6 public function index() {
7     return view('estudiantes.index', [
8         'estudiantes' => Estudiante::all(),
9     ]);
10 }
11 // La vista Blade recorre $estudiantes y escapa cada dato.

```

Symfony — listar estudiantes

```

1 <?php
2 #[Route('/estudiantes', name: 'estudiantes_index')]
3 public function index(EstudianteRepository $repo): Response {
4     return $this->render('estudiantes/index.html.twig', [
5         'estudiantes' => $repo->findAll(),
6     ]);
7 }
8 // La plantilla Twig recorre los estudiantes y escapa cada dato.

```

Desaparecieron la conexión manual, el `htmlspecialchars` por celda y el manejo del método HTTP: el framework se encarga. A cambio, debes aprender sus convenciones y su estructura. Por eso era importante construirlo antes a mano: ahora entiendes qué hace el framework por ti.

12.9 Tabla comparativa y ¿cuál elegir?

Esta tabla resume las diferencias prácticas que más te ayudarán a decidir. Ninguna opción es incorrecta; la elección depende del proyecto, del equipo y de los plazos.

Aspecto	Laravel vs Symfony
Filosofía	Convención sobre configuración / Componentes y configuración explícita.
ORM	Eloquent (Active Record) / Doctrine (Data Mapper).
Plantillas	Blade / Twig (ambas escapan por defecto).
Rutas	Archivo central + helpers / Atributos en el controlador.
Consola	artisan / symfony + MakerBundle.
Curva	Más suave; ideal para empezar / Más pronunciada; muy didáctica en arquitectura.
Casos de uso	MVP, startups, APIs rápidas / Sistemas grandes y de larga vida, empresariales.
Relación	Laravel usa por debajo varios componentes de Symfony.

Recomendación para estudiantes: empieza por **Laravel**, porque verás resultados muy rápido y refuerza los conceptos con poca fricción. Conoce **Symfony** porque te enseña una arquitectura sólida y domina el ámbito empresarial. Lo esencial —MVC, ORM, rutas, plantillas, validación— es el mismo en ambos y, además, se transfiere casi directamente a Spring Boot, el framework de tu Proyecto Fin de Curso en Java.

No empieces por el framework

El error más común es saltar directo a un framework sin entender PHP. Si llegaste hasta aquí dominando los fundamentos de esta guía, ahora un framework te potenciará; si lo usas como una caja negra, cada error te bloqueará. Primero los cimientos, luego la herramienta.

12.10 Ejercicios — frameworks

Estos ejercicios te invitan a dar el primer paso real con un framework y a comparar enfoques. Necesitarás Composer instalado.

Da el salto al framework:

1. **[Intermedio]** Instala Laravel, crea una ruta `/hola` que devuelva un saludo y otra `/hola/{nombre}` que lo personalice.
2. **[Intermedio]** En Laravel, crea el modelo **Estudiante** (Eloquent) y una vista Blade que liste todos los estudiantes de la base.

3. **[Avanzado]** Replica el CRUD del proyecto integrador (sección 9) en Laravel: rutas, controlador, Eloquent y Blade.
4. **[Avanzado]** Haz el mismo listado en Symfony con Doctrine y Twig, y compara cuántas líneas te tomó frente a Laravel y frente a la versión a mano.
5. **[Avanzado]** Redacta tu propia tabla comparando cómo defines una ruta y cómo accedes a los datos en tres enfoques: PHP a mano, Laravel y Symfony.

13 Banco de ejercicios por nivel

Este banco reúne ejercicios adicionales para que practiques de forma intensiva. Están agrupados por nivel; al final de la guía encontrarás soluciones comentadas de algunos de ellos. Resuélvelos en tu entorno y verifica con casos de prueba.

13.1 Nivel básico

Refuerzan la sintaxis, los tipos, los arrays y las funciones. Deberías resolver cada uno en pocos minutos.

1. Muestra la tabla de multiplicar de un número recibido por GET.
2. Cuenta cuántas vocales tiene una cadena dada.
3. Invierte un array sin usar `array_reverse` (con un bucle).
4. Dado un array de números, devuelve solo los pares con `array_filter`.
5. Escribe una función que reciba una temperatura en Celsius y la devuelva en Fahrenheit.
6. Genera un array asociativo con cinco productos y sus precios, y muestra el más caro.
7. Determina si una palabra es un palíndromo.
8. Calcula el factorial de un número con una función recursiva.

13.2 Nivel intermedio

Combinan formularios, validación, sesiones y objetos. Se parecen a tareas reales de una aplicación.

1. Formulario de contacto con validación de nombre, correo y mensaje, que muestre todos los errores juntos.
2. Conversor de divisas con un select de monedas y `match` para las tasas.
3. Lista de tareas (to-do) que guarde las tareas en sesión y permita marcarlas como hechas.
4. Clase `Carrito` que sume productos y calcule el total con IVA.
5. Paginación de un array grande de datos (mostrar 10 por página con `?pagina=`).
6. Subida de un archivo de imagen con validación de tipo y tamaño (`$_FILES`).
7. Clase `Validador` reutilizable con métodos `requerido`, `correo` y `entero`.

13.3 Nivel avanzado

Exigen diseño, acceso a datos y seguridad. Son los más cercanos a un proyecto profesional.

1. API mínima en PHP que devuelva los estudiantes en formato JSON (`header('Content-Type: application/json')`).
2. CRUD completo de «productos» con PDO, validación, PRG y protección CSRF.
3. Sistema de login con roles (admin/usuario) que restrinja páginas según el rol guardado en sesión.
4. Repositorio con inyección de PDO y pruebas unitarias de su lógica con PHPUnit.
5. Buscador con filtros combinados (carrera y texto) usando sentencias preparadas dinámicas.
6. Reporte que exporte los datos a CSV descargable con las cabeceras adecuadas.

13.4 Proyectos propuestos

Para integrar todo en algo más grande, elige uno de estos proyectos y desarróllalo por etapas, aplicando seguridad desde el inicio.

1. Agenda de contactos con CRUD, búsqueda y autenticación.
2. Mini blog con entradas, comentarios (escapados) y panel de administración protegido.
3. Sistema de reservas (aulas u horarios) con validación de solapamientos en transacciones.
4. Gestor de calificaciones con reportes por estudiante y por asignatura.

14 Soluciones seleccionadas

Aquí tienes soluciones comentadas de algunos ejercicios del banco para que contrastes tu enfoque. Recuerda que casi siempre hay más de una solución válida; lo importante es que sea correcta, legible y segura.

14.1 Vocales en una cadena (básico 12.1.2)

Recorremos la cadena y contamos los caracteres que son vocales. Convertimos a minúsculas para no distinguir mayúsculas.

Solución: contar vocales

```

1 <?php
2 function contarVocales(string $texto): int {
3     $texto = mb_strtolower($texto);
4     $n = 0;
5     foreach (str_split($texto) as $c) {
6         if (in_array($c, ['a','e','i','o','u'], true)) {
7             $n++;
8         }
9     }
10    return $n;
11 }
12 echo contarVocales('Murcielago'); // 5

```

14.2 Pares con array_filter (básico 12.1.4)

`array_filter` conserva los elementos para los que la función devuelve verdadero. Usamos una función flecha que comprueba el resto de la división entre dos.

Solución: números pares

```

1 <?php
2 $nums = [1, 2, 3, 4, 5, 6];
3 $pares = array_filter($nums, fn(int $n) => $n % 2 === 0);
4 print_r(array_values($pares)); // [2, 4, 6]

```

14.3 API JSON de estudiantes (avanzado 12.3.1)

Una API devuelve datos en lugar de HTML. Consultamos con PDO, fijamos la cabecera de tipo JSON y codificamos el array con `json_encode`.

Solución: API JSON

```

1 <?php
2 declare(strict_types=1);
3 require __DIR__ . '/config/db.php';
4
5 header('Content-Type: application/json; charset=utf-8');
6 $datos = $pdo->query('SELECT id, nombre, correo, carrera FROM estudiantes ORDER BY id')
7     ->fetchAll();
8 echo json_encode($datos, JSON_UNESCAPED_UNICODE);

```

Pruébala abriéndola en el navegador o con una herramienta como `curl`: deberías ver un arreglo JSON con los estudiantes. Es la base de un backend que luego consume un frontend (por ejemplo, en Angular).

15 Glosario de siglas y términos

Este glosario reúne las siglas y términos que aparecen en la guía, para consulta rápida durante el estudio.

Término	Significado
PHP	PHP: Hypertext Preprocessor (acrónimo recursivo).
HTTP/HTTPS	HyperText Transfer Protocol / Secure.
HTML	HyperText Markup Language.
CSS	Cascading Style Sheets.
PDO	PHP Data Objects (capa de acceso a datos).
DSN	Data Source Name (cadena de conexión).
SQL	Structured Query Language.
CRUD	Create, Read, Update, Delete.
OOP	Programación Orientada a Objetos.
PSR	PHP Standards Recommendation.
Composer	Gestor de dependencias de PHP.
XSS	Cross-Site Scripting.
CSRF	Cross-Site Request Forgery.
PRG	Post-Redirect-Get.
JSON	JavaScript Object Notation.
Superglobal	Array predefinido accesible desde cualquier ámbito.
OWASP	Open Web Application Security Project.

16 Recursos y referencias

Para seguir aprendiendo y consultar dudas, estos son los recursos de referencia más fiables del ecosistema PHP.

- Manual oficial de PHP: **php.net/manual**
- Estándares de la comunidad (PSR): **php-fig.org**
- Composer y paquetes: **getcomposer.org** y **packagist.org**
- PostgreSQL: **postgresql.org/docs**
- Seguridad web (OWASP Top 10): **owasp.org/Top10**
- Buenas prácticas para principiantes: **phptherightway.com**

Con esta guía y la práctica constante de los ejercicios, tendrás una base sólida para desarrollar aplicaciones web con PHP y para dar el salto, más adelante, a un framework como Laravel o Symfony.